AwaazIQ (ComplainBox)

Complete Project Audit

From-source, feature-by-feature audit of the entire codebase — compiled and verified 2026-09-16, branch `village-priority-system`

Compiled 2026-09-16, on branch `village-priority-system`, by reading the actual source files listed under each section — not summarized from an old document. Every claim below was checked against `backend/models.py`, `intelligence/*.py`, `backend/routes_*.py`, `pipeline/*.py`, the `backend/load_*.py` scripts, and the project's own `README.md`, `FEATURE_ROADMAP.md`, `features.md`, `intelligence/WEIGHTS.md` and `REAL_DATA_RESEARCH.md`.

Updated same day, later pass: the `GET /citizen/my-reports` crash this audit's own Part 10 documented (item 1) has since been fixed and verified live — see that section for the fix detail, kept rather than deleted so the audit still shows its own finding led somewhere. This pass also adds Part 15, on deployment readiness for Vercel (frontend) + Railway (backend) + Supabase (Postgres), none of which existed as a real target when the first pass was written.

This supersedes an earlier PDF study document that was built against a much smaller, older snapshot of this codebase (no photo verification, no village/asset view, no emergency-urgency term, no budget optimizer, roughly a third of today's data loaders). That document is not reproduced or trusted here — everything below is re-derived from the code as it exists today, and every place where this document found the old one (or the project's own `README.md`) to have been *wrong* is called out explicitly, because this project has a habit of catching and correcting its own earlier mistakes and that habit is worth preserving here too.

---

0. At a glance (2026-09-16)

Database	SQLite, `backend/hackathon.db`, WAL mode
`gazetteer`	1,042 places, Kolhapur + Nashik districts, Maharashtra
Named facilities	10,464 (9,352 UDISE schools + 1,112 health facilities)
PMGSY sanctioned road works	552
`citizen_request` rows	2,099 (1,000 flagged `is_synthetic`, the rest real/test intake — grows as testing continues)
↳ assigned to a cluster	1,502
`demand_cluster` / `priority_score` rows	118 — priority scores range 18.69–56.89
Specific assets ranked	819 assets across 443 villages
Photo checks (Workstream C)	C1–C7 built, including a live vision-LLM damage model (C6)
Test coverage	intelligence 291/291 · backend endpoint 70/70 · pipeline 46/46 · photo checks 65/65
Real (non-proxy) data sources loaded	20 loader scripts, ~15 genuinely live/current government sources
Backend + intelligence + pipeline code	~21,500 lines across `backend/`, `intelligence/`, `pipeline/` (excluding tests, scratch scripts, and the 190 MB `backend/data/`)

What this project actually is, in one paragraph: citizens file infrastructure complaints (road/water/health/education) by text or voice, in Hindi, Marathi, English or a mix, optionally with photos. A rule-based NLP pipeline extracts category, severity and location; a clustering pass groups corroborating reports into named, geolocated **assets** (this specific school, this specific stretch of road) and rolls those up into **villages**; a nine-term, fully-decomposable scoring formula ranks every asset from 0–100 using a mix of real government records (Census, PMGSY, JJM, UDISE+, BharatNet, MGNREGA, PRIASoft, NDEM flood data, CWC river levels, SACHET alerts, NWDP groundwater) and clearly-labelled proxies where no real data exists yet. Attached photos are run through seven independent authenticity/defect checks (Workstream C) that never reject a complaint, only adjust confidence and flag doubtful evidence for a human. Everything is queryable through a FastAPI backend and rendered on a D3 map dashboard.

PART 1 — Problem, thesis, and what changed since the last audit

1.1 The problem

India's existing grievance system (CPGRAMS) routes each citizen complaint individually to a department. There is no aggregation of similar complaints, no comparison against what the government has already sanctioned or spent nearby, and — most importantly to this project's own design — **no correction for the fact that people in poorly-connected, remote areas report less, not because they need less, but because reporting is harder for them.**

1.2 The thesis, and the number that proves it

> *A quieter cluster in an underserved area can and does outrank a louder one in a well-connected area, because the score corrects for who can afford to complain.*

Verified on live data (`README.md`): Cluster 50 (road, Chandgad — a low-connectivity block) scored **62.05** on 4 citizen reports; Cluster 88 (education, Kagal — not low-connectivity) scored **45.37** on 11 reports. The equity term (+10.0 for Chandgad) and the demand term's 25-reporter saturation cap are what carry it, and both numbers are visible in the score breakdown, not hidden inside a model.

1.3 What this audit found that the project's own last full study document (the old PDF)

never saw

The repo has grown enormously since that document was produced. Confirmed by `git log` and by reading the actual current files, all of the following are **new since that snapshot** and are documented in full later in this audit:

- An entire **village → asset priority view** (`VillagePriority`, `Asset` tables) that replaced "demand cluster" as the dashboard's primary unit — a village's score is now the score of its single worst-off named asset (a specific school, a specific stretch of road), not an abstract cluster centroid.
- **Workstream C — photo verification and fake-complaint defence** (`backend/photo_checks.py`, 1,027 lines): seven independent checks (capture provenance, duplicate-photo hashing, edit detection, road-defect detection via a real YOLOv8 model, a vision-LLM-based building/bridge damage grader, and a screen-replay/moiré check), none of which ever rejects a complaint.
- **Emergency Urgency** (`intelligence/emergency.py`) — a complete redesign of the old flat "+3 points for road/water" urgency term into a graded (crack/partial/collapse), multi-signal-confidence bridge/building emergency detector.
- A **0/1 knapsack budget optimizer** (`intelligence/budget_optimizer.py`) that replaced the old flat "what-if" re-scoring simulation with real constrained optimisation over actually -costed road/water projects.
- A **stale-record trust-blend** in the scoring engine: government records now lose trust exponentially with age when contradicted by a real burst of recent, high-severity citizen reports (`record_freshness()`, `RECORD_TRUST_HALF_LIFE_YEARS = 10.0`).
- **Eleven additional real government/near-real-time data sources** loaded and wired as evidence since the old document: BharatNet fibre status is now *proportional*, not a 10-block hardcoded list (though that list still exists as the fallback); UDISE+ Know Your School per-school 2024-25 condition and grant data; JJM water-testing coverage (AES -encrypted site, defeated); JJM per-scheme cost/status; MGNREGA panchayat expenditure; PRIASoft village-panchayat receipts/expenditure; eGramSwaraj GPDP district summaries; NWDP groundwater telemetry; CWC live river-level readings; SACHET live disaster alerts; and NDEM satellite-derived historical flood-inundation extents.
- **Precise citizen GPS pins**, specific-facility selection on the intake form, and PMGSY GeoSadak real road-line geometry for naming and drawing road assets.

1.4 What this audit is *not* going to repeat uncritically

Two things the earlier document asserted turned out to be wrong when checked against the live system, and the project's own `README.md` already documents this self-correction — this audit follows the current, corrected version, not the older claim:

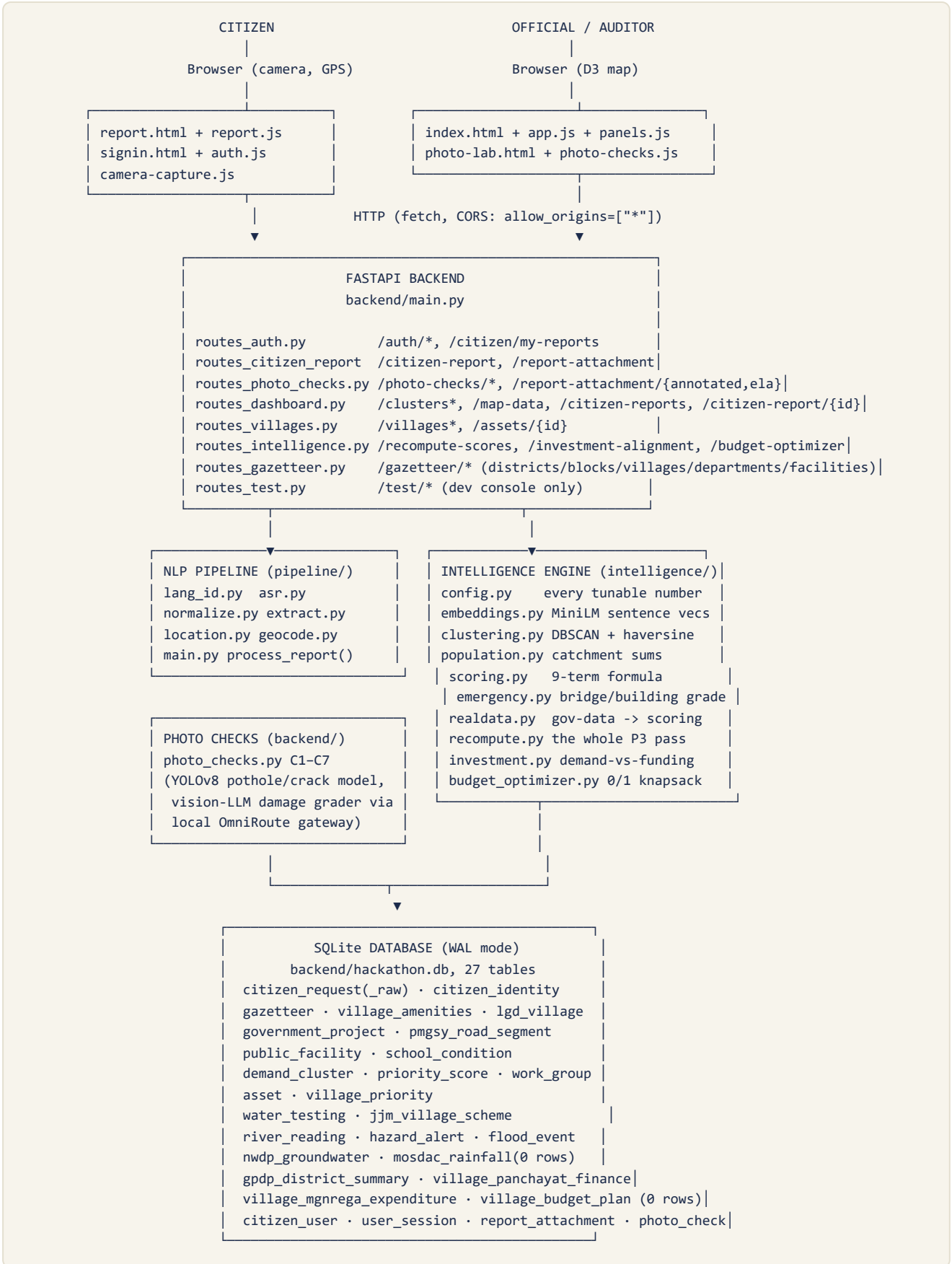
- `language_detected` values are exactly the contract's `hi-Deva` / `mr-Deva` / `hi-Latn` / `mixed` / `en` codes — an earlier README claimed the opposite; it was wrong.
- `extract_issue_category` choosing "most keyword hits" (not "first category with any hit") was claimed fixed once already and was not — the fix had been lost in a branch merge and had to be redone. Reading the current `pipeline/extract.py` (quoted in full in Part 6.1) confirms the fix is genuinely present now.

PART 2 — Complete system workflow, end to end

```
CITIZEN (report.html, live camera, GPS)
   │  POST /citizen-report  (JSON {"text": "..."} OR multipart intake form
   │                         with photos, capture_meta, burst_<i> frames)
   ▼
backend/routes_citizen_report.py :: create_citizen_report()
   │  - decides JSON-ingest vs full CPGRAMS-style intake form
   │  - validate_intake(): name/gender/mobile/email/address/pincode/
   │    district/block/village/department (read off the signed-in
   │    account when one exists, never re-typed)
   ▼
pipeline/main.py :: process_report()
   │  1. audio? -> pipeline/asr.py transcribe() (faster-whisper, hi/mr/en only)
   │  2. pipeline/lang_id.py detect_language()   -> hi-Deva/mr-Deva/hi-Latn/mixed/en/+8 others
   │  3. pipeline/normalize.py normalize_text()  -> strip fillers, canonicalise transliteration
   │  4. pipeline/extract.py extract_issue_category() -> road/water/health/education (most-hits wins)
   │  5. pipeline/extract.py extract_severity()  -> high/medium/low
   │  6. pipeline/location.py extract_location_phrase() -> best-scoring trigger window
   │  7. pipeline/geocode.py resolve_location()  -> rapidfuzz WRatio against 1,042-row gazetteer
   │  8. confidence decay across the 4 field scores
   ▼
routes_citizen_report.py (continued)
   │  - redact_pii(): strip Indian mobile numbers + self-stated names from the ANALYTICAL
   │    copy only (citizen_request.raw_text); the untouched original goes to
   │    citizen_request_raw, and full CPGRAMS identity goes to citizen_identity — a table
   │    NOTHING in the dashboard/analytics path ever joins to.
   │  - if a full intake form: the citizen's PICKED village overrides whatever the pipeline
   │    guessed from free text, and the recovered issue_category is backfilled from the
   │    chosen department if the text pipeline failed to classify it
   │  - a citizen-supplied GPS pin (precise_lat/precise_lon) is validated and stored
   │    alongside the village centroid (latitude/longitude), never replacing it
   │  - save_attachments(): photos/PDFs, CPGRAMS limits (5 files, 4 MB each), stored under a
   │    random token name — the uploader's filename never touches the filesystem path
   ▼
backend/routes_photo_checks.py :: check_report_photos()  [Workstream C]
   │  For every attached photo, runs backend/photo_checks.py :: analyze_photo() —
   │  C1 capture provenance, C3 duplicate-photo hash, C4 error-level analysis,
   │  C5 pothole/crack detector (YOLOv8), C6 bridge/building damage grade
   │  (vision-LLM), C7 screen-replay (moiré + burst liveness).
   │  Nudges (never zeroes) citizen_request.confidence and photo_trust.
   ▼
DATABASE WRITE
   citizen_request (redacted text + structured fields + PII-free)
   citizen_request_raw (untouched original text)
   citizen_identity (name/mobile/email/address — never joined by analytics)
   report_attachment (file metadata; bytes under backend/uploads/)
   photo_check (one row per photo, full C1–C7 working)
   ▼
[LATER — POST /recompute-scores, or python -m intelligence.recompute]
   ▼
intelligence/recompute.py :: recompute()
   │  1. Load every citizen_request + gazetteer + EVERY real-data index (11 of them —
   │     amenities, PMGSY works, GeoSadak road segments, groundwater stations, JJM water
   │     testing, CWC river readings, SACHET alerts, UDISE+ school condition, JJM per-scheme
   │     records, MGNREGA expenditure, NDEM flood events) — loaded ONCE, reused for every
   │     cluster/asset, never re-queried per row.
   │  2. cluster_all(): DBSCAN per issue_category over a combined geo+semantic distance
   │     matrix (intelligence/clustering.py); noise points stay unclustered on purpose.
   │  3. _attach_stragglers(): a second pass that folds later lone reports into an
   │     ALREADY-established cluster of the same category within the gate radius — this
   │     never creates a new cluster, so DBSCAN's min_samples=3 corroboration rule is not
   │     weakened.
   │  4. _score_members() -> intelligence/scoring.py :: score_cluster() for every cluster
   │     AND for every asset — the 9-term Gap Score + Priority Score formula, real data
   │     wherever intelligence/realdata.py has it, honest proxies (with a confidence
   │     penalty) everywhere else.
   │  5. _build_assets(): resolves each cluster's member reports down to a SPECIFIC named
```

```
   │     thing — a citizen-picked school/hospital, a PMGSY-named road, a GeoSadak-matched
   │     road segment, or (honestly) an "unresolved" bucket with a candidate shortlist.
   │  6. _build_villages(): rolls every village's assets up into one VillagePriority row —
   │     the village's score is its single highest-scoring asset's score, plus an
   │     INDEPENDENT has_urgent_asset flag so a real emergency at a lower-ranked asset in
   │     the same village is never hidden behind a higher-scoring but non-urgent one.
   ▼
DASHBOARD (frontend/index.html + app.js + panels.js)
   GET /villages?district=      -> village bubbles, sized/coloured by priority + urgency
   GET /villages/{gazetteer_id} -> village panel: every asset, top asset breakdown, reports
   GET /assets/{id}             -> asset panel: 9-term breakdown, evidence, photos, reports
   GET /clusters, /clusters/{id}, /clusters/{id}/reports -> the older cluster-level view,
       still live and still the unit /map-data and the priority rail render by default
   GET /map-data?layer=clusters|reports -> GeoJSON
   GET /investment-alignment    -> demand vs government funding (roads + schools)
   GET /budget-optimizer        -> 0/1 knapsack allocation for a given budget
   POST /recompute-scores       -> re-run the whole pass (idempotent, ~1 min on today's data)
   ▼
OFFICIAL sees a ranked, evidence-backed, provenance-labelled worklist and can drill into
exactly why any one asset scored what it did, down to the individual government record.
```

PART 3 — Architecture



No external service is called at request time except two: the sentence-embedding model runs in-process (downloaded once, cached), and the C6 vision-LLM damage grader calls a local OmniRoute gateway subprocess (`backend/photo_checks.py :: vlm_grade_damage()`), which is this machine's own convenience for the prototype and is explicitly documented as needing to be swapped for a direct provider API key before real deployment. Every `load_*.py` script is a one-shot job run by hand to populate the database;

nothing in the request path re-fetches government data live except the SACHET/CWC loaders, which are themselves one-shot scripts a human re-runs, not a scheduled job.

PART 4 — Technology stack

Layer	Technology	Where / why
Backend	Python 3, FastAPI, SQLAlchemy, SQLite (WAL mode, `PRAGMA foreign_keys=ON`)	`backend/main.py`, `backend/database.py`
NLP	Rule-based keyword dictionaries + regex (no ML, no LLM)	`pipeline/lang_id.py`, `extract.py`, `location.py`
Speech-to-text	`faster-whisper` (`small` model, CPU, int8)	`pipeline/asr.py`
Fuzzy geocoding	`rapidfuzz` (`fuzz.WRatio`, threshold 75)	`pipeline/geocode.py`, and 7+ data loaders that fuzzy-match village names
Sentence embeddings	`sentence-transformers`, `paraphrase-multilingual-MiniLM-L12-v2`, 384-dim, in-memory	`intelligence/embeddings.py`
Clustering	`scikit-learn` DBSCAN, `metric="precomputed"`	`intelligence/clustering.py`
Numerics	`numpy`	distance matrices, image arrays
Object detection	Ultralytics YOLOv8 (team-trained, `models/pothole_best.pt`, 25.9M params)	`backend/photo_checks.py`
Building/bridge damage grading	Vision-LLM via local OmniRoute gateway (`antigravity/gemini-3.7-flash-medium` by default)	`backend/photo_checks.py :: vlm_grade_damage()`
Image processing	Pillow, `imagehash` (perceptual hashing), `scipy.ndimage` (moiré spectrum)	`backend/photo_checks.py`
Frontend	Vanilla HTML/CSS/JS, no framework	`frontend/`
Map rendering	D3.js v7 + TopoJSON v3	`frontend/js/app.js` (1,996 lines)
Live camera capture	`getUserMedia` + ImageCapture API, `navigator.geolocation.watchPosition`	`frontend/js/camera-capture.js`
Web scraping (data loaders)	`requests`, `urllib`, `BeautifulSoup`, `pycryptodome` (AES), `py7zr`, `pyarrow` (parquet), `playwright` (GPDP capture only)	`backend/load_*.py`
Auth	PBKDF2-SHA256 (100,000 iterations, random 16-byte salt), opaque URL-safe session tokens	`backend/routes_auth.py`

PART 5 — Database schema, table by table (from `backend/models.py`, 986 lines, 27 tables)

5.1 The citizen-facing tables

`citizen_request` — the analytical record. `raw_text` (PII-redacted), `language_detected`, `issue_category`, `severity`, `location_raw`, `district` / `block` / `village`, `latitude` / `longitude` (village centroid, always set on a resolved report), `precise_lat` / `precise_lon` (nullable — the citizen's actual GPS pin, added later; the centroid columns are never overwritten by it), `confidence`, `is_synthe

`tic`, `department`, `cluster_id` (FK, null until clustered or if DBSCAN calls it noise), `user_id` (FK, null if anonymous), `facility_id` (FK to `public_facility` — the specific school/hospital a citizen picked), `pin_source` (`"citizen_gps"` or `"synthetic_seed"`), `asset_id` (FK, set by every recompute), `photo_trust` (float, worst photo authenticity across this report's photos, null if none), `review_flags` (JSON list of photo-check flags needing an officer's eye).

`citizen_request_raw` — the untouched original text, in a separate table, never shown on the dashboard. Legal/audit record.

`citizen_identity` — CPGRAMS-level identity: `full_name`, `gender`, `mobile`, `email`, `address`, `pincode`, `state`. **Deliberately never joined by any analytics or dashboard route** — enforced by the fact that no dashboard route file even imports `CitizenIdentity`, not by a runtime permission check. This is the project's core privacy design: identity is collected (a real grievance needs someone behind it) but structurally unreachable from the path that decides where money goes.

`report_attachment` — file metadata only. `stored_name` is a random 32-hex-char token plus an extension derived from the *declared content-type*, never from the uploader's filename — so a crafted filename like `../../etc/passwd.jpg` cannot escape the upload directory or spoof an executable extension.

`citizen_user` / **`user_session`** — registered citizen accounts (PBKDF2 password hash, opaque bearer token, no expiry currently enforced — `expires_at` exists but nothing ever sets it).

5.2 Geography and reference data

`gazetteer` — 1,042 rows: `name`, `admin_level`, `district`, `block`, `population` (Census 2011, 948/1,042 rows real), `latitude` / `longitude` (OpenStreetMap). `block` is a **geometric approximation** (nearest taluka HQ within the same district), not an official boundary — flagged as such everywhere it's used, and now cross-checked against `lgd_village`'s real hierarchy (58% taluka agreement, 48% development-block agreement — genuinely a different, third thing).

`lgd_village` — the *official* Local Government Directory hierarchy: taluka (subdistrict) and development block stored **separately** (they are genuinely different administrative units in India, and collapsing them into one `gazetteer.block` field is exactly what produced the ambiguity above). Carries Census 2011 codes at every level — the join key this project never had until this table existed.

`village_amenities` — the real Census 2011 District Census Handbook (Village Amenities schedule) record for a village, one row per matched `gazetteer_id`: road surface type, water-tap type, health facility counts, school counts, connectivity, isolation, drainage, power hours, plus current (2022+) BharatNet fibre status and current JJM tap-connection coverage layered on top of the same row. Status columns use the Census's own `1`=available, `2`=not available, `NULL`=unknown encoding — unknown is never collapsed into 0.

`public_facility` — named, located assets: 9,352 UDISE schools + 1,112 NIC health facilities, each with `source`, `external_id` (UDISE code), `category`, `sub_type`, `management`, coordinates.

`school_condition` — real, current (2024-25) per-school UDISE+ Know Your School data: teacher counts by type, classroom condition (good/minor repair/major repair), functional toilets, electricity, drinking water, total grant and total expenditure. `fetch_failed` marks schools that could not be fetched after retries so "no row" is never confused with "verified fine".

`government_project` — real, individually-sanctioned PMGSY road works: sanction year, cost, length, agreement details, `work_status` (only ever "Not Started" / "In Progress" / "Agreement Cancelled" — **never "Completed"**, so this table is the backlog, not the whole programme, and must never be summed against the district aggregate's Balance figure). `matched_gazetteer_id` nullable — only ~33% of works are pinned to a known village by name match; unpinned works are kept, not dropped.

`pmgsy_road_segment` — real line geometry (points list, bounding box) from PMGSY GeoSadak's Road_DRRP layer: official road name, DRRP road code, category (`RR(VR)`, `MDR`, `SH`, `NH`…), owning agency (`RWD`, `PWD`, `MRRDA`…). No surface/condition attributes exist in this layer — it is identity and location only, not a condition source.

5.3 The scoring/output tables (rebuilt on every recompute)

`demand_cluster` / **`priority_score`** — the older, still-live cluster-level unit: centroid, report count, unique reporters, population affected, district/block (modal), `avg_confidence`, `settlement_count`. `priority_score.breakdown` (JSON, 9 terms summing exactly to the score) and `priority_score.evidence` (JSON, the full working behind every term) are recomputed and overwritten every pass. `runner_up_cluster_id` names the next-best cluster for free "why not this one instead" comparison.

`work_group` — a sub-cluster unit: reports within `WORK_GROUP_RADIUS_M` (250 m) of each other, single-linkage grouped, so a cluster spanning several villages can still say which specific spot inside it is broken. `asset_label` is set **only** when a facility/road match is unambiguous; otherwise `asset_candidates` (JSON shortlist) is returned and a human decides.

`asset` — the newer, primary unit the village view is built on: a specific named thing (a school, a hospital, a PMGSY-named road, a GeoSadak-matched road segment, or an honestly labelled "unresolved" bucket). Carries its own full 9-term `breakdown` and `evidence` JSON, `name_basis` (how it got its name: `citizen_selected` / `demo_assigned` / `nearest_register` / `geosadak_segment` / `pmgsy_work` / `unnamed_pin` / `unresolved_village`), `location_basis` (`register_coordinates` / `citizen_gps_pin` / `synthetic_seed` / `village_centroid` / `mixed`), `is_demo` (true only when every member report is synthetic).

`village_priority` — one row per village that has any report: `priority_score` and `top_asset_id` are always the village's single highest-scoring asset (a standing design decision — a road score and a hospital score in the same village are never summed). `has_urgent_asset` / `urgent_asset_id` / `urgent_grade_label` are **computed independently** of that ranking, specifically so a real photographed emergency at a lower-scoring asset in the same village is never hidden behind a higher-scoring but non-urgent one (this exact failure mode was found live on 2026-09-16 — a roof collapse at a school hidden behind a higher-scoring road in the same village — and this flag is the fix). `rank_in_district` computed per district.

5.4 Real-data-corroboration tables (evidence-only — none of these move a score directly

except via `realdata.py`'s named functions feeding `score_cluster()`)

`water_testing` (JJM WQMIS current testing coverage — pH/FRC/turbidity/TDS/hardness sample counts, not contamination readings), `jjm_village_scheme` (per-scheme cost/status), `river_reading` (live CWC gauge readings), `hazard_alert` (live SACHET CAP alerts), `flood_event` (14,434 real NDEM 2013/2021 flood-inundation bounding boxes), `nwdp_groundwater` (live DWLR telemetry), `mosdac_rainfall` (**table exists, 0 rows — correctly left empty**, see Part 8.4), `gpdp_district_summary` (district-level GPDP planning totals — explicitly *not* wired into any village/asset score, since a district total cannot be honestly split down), `village_panchayat_finance` (real PRIASoft receipts/expenditure, tied/untied 15th Finance Commission grants), `village_mgnrega_expenditure` (real MGNREGA panchayat spending), `village_budget_plan` (eGramSwaraj GPDP per-village line items — **table exists, 0 rows**, the human-assisted capture tool was built but never actually run, see Part 8.4).

`photo_check` — one row per photo attachment: capture provenance (method, timestamp, GPS + accuracy, distance to village — coordinates stored but never returned by a public endpoint), perceptual hashes (`phash` / `dhash`), error-level score, screen-replay score, pothole/crack confidences, damage grade/confidence/structure, combined `authenticity`, `verdict`, `flags` (JSON), and the full `result` JSON for an officer to audit.

PART 6 — Feature-by-feature breakdown

6.1 Citizen complaint ingestion — text and voice

Files: `backend/routes_citizen_report.py` (727 lines), `pipeline/main.py`, `pipeline/lang_id.py`, `pipeline/normalize.py`, `pipeline/extract.py`, `pipeline/location.py`, `pipeline/geocode.py`, `pipeline/asr.py`.

Two request shapes hit `POST /citizen-report`:

1. **Bare JSON** `{"text": "..."}` — the original ingest path, no identity, used by the seeding scripts and the endpoint tests. Unchanged since the project's earliest version.
2. **`multipart/form-data`** — the real citizen intake form. Sending `full_name`, `village` or `department` switches on full CPGRAMS-style validation (`validate_intake()`), which requires the whole field set. If the citizen is signed in, **identity is read off their account, never re-typed** — this is a deliberate anti-duplicate-voter measure: a citizen who retypes their name slightly differently on their second complaint would otherwise be counted as two corroborating people rather than one.

Step-by-step, exactly as implemented (`pipeline/main.py :: process_report()`):

Step	Function	What it does	Confidence contribution
1	`asr.transcribe()` if audio	faster-whisper, `small`, restricted to hi/mr/en, VAD-filtered, no repeat loops, per-language domain prompt	—
2	`lang_id.detect_language()`	Unicode-range + marker-word rules → `hi-Deva` / `mr-Deva` / `hi-Latn` / `mixed` / `en`, or one of 8 other Indic scripts flagged unsupported	0.95/0.95/0.80/0.70/0.90, or 0.15 if unsupported
3	`normalize.normalize_text()`	Strip filler words (`umm`, `matlab`, `मतलब` …), canonicalise transliteration (`sadak` → `road`, `paani` → `water` …)	—
4	`extract.extract_issue_category()`	Count keyword hits per category across 4 language variants; **the category with the most hits wins, declaration order only breaks true ties** — this exact bug ("first category with any hit wins") was found, fixed, silently re-broken by a branch merge, and fixed again; reading the current file confirms the fix is genuinely in place today	0.90 if found, else 0.30
5	`extract.extract_severity()`	High/low keyword tiers across languages, or `duplicate_count > 5` → high; default medium	0.90/0.85/0.70
6	`location.extract_location_phrase()`	Scores **every** trigger-word window (not just the first) and keeps the one containing an actual place-like token — the classic "Hamare gaon mein road bahut kharab hai, Karvir ke paas" case, where the first trigger ("gaon") sits nowhere near the real place name ("Karvir")	geocoding confidence below
7	`geocode.resolve_location()`	`rapidfuzz.process.extractOne(..., scorer=fuzz.WRatio)` against the 1,042-row gazetteer, threshold 75/100	match score ÷ 100, or 0.20 if unresolved
8	`_apply_confidence_decay()`	Average the four scores above; if more than half are below 0.60, apply an extra ×0.90 penalty	final `confidence_overall`

If the form path supplied a picked village, `resolve_picked_location()` **overrides** whatever the text pipeline guessed (a citizen picking from a dropdown cannot produce an unresolvable location), `location_raw` is replaced with the picked village name, and confidence is boosted toward 1.0 by half its remaining gap — because location is no longer a source of doubt once it's a dropdown choice.

If the text pipeline still returns no `issue_category` (common for heavily code-switched or Latinized text, e.g. "Chat khrab hai bhot"), and the form supplied a `department`, the category is recovered from which department the citizen picked (`category_for_department()` in `routes_photo_checks.py`) — otherwise the report would carry `issue_category = None` forever and `intelligence.clustering.cluster_all()` silently drops any report with no category, so it could never be clustered no matter how many times a recompute ran.

6.2 PII redaction

File: `backend/routes_citizen_report.py :: redact_pii()`.

Two regex families run against the analytical copy of the text only (never the raw copy in `citizen_request_raw`, never `citizen_identity`):

- `PHONE_RE` — a 10-digit Indian mobile number starting 6–9 → `[PHONE]`.
- `NAME_PREFIX_RE` — a title followed by a capitalised word (`Mr` / `Mrs` / `Ms` / `Miss` / `Dr` / `Shri` / `Smt`) → `[NAME]`.
- `SELF_NAME_RE` — self-identification phrasing: "mera naam Ravi Patil", "my name is Ravi", "माझं नाव रवी". The possessive ("mera"/"my"/"माझं") is **required** in the pattern on purpose — matching a bare "naam"/"नाव" would also eat "गावाचे नाव Bidri" ("the village's name is Bidri"), and the village name is the one thing the clustering embedding genuinely needs to keep.

This is explicitly *not* a general-purpose name detector — the code's own comment says "no regex can be one" — it removes only the forms people actually write in these complaints.

6.3 Speech-to-text (voice complaints)

File: `pipeline/asr.py`.

Constrained to `hi` / `mr` / `en` by reading Whisper's own per-language probability distribution and taking the best *supported* one (falling back to Hindi below a 0.25 confidence floor), rather than letting vanilla Whisper choose across 99 languages — unconstrained detection had been reading Hindi speech as English and then **translating** it instead of transcribing, because Whisper only translates once it decides the audio is English. `vad_filter=True` strips silence (Whisper's best-documented hallucination, "Hello, how are you?", happens over silence), `condition_on_previous_text=False` stops repeat-loop self-echoing, and a short per-language domain-vocabulary prompt (road/pothole/water/borewell/hospital/school/village terms) biases decoding toward civic vocabulary.

Model size is `small` by explicit measured decision, not default laziness: one TTS clip per language showed `medium` is ~2× slower, worse on Hindi ("गाव"/"हैं" errors), and *still wrong* on Marathi. **Marathi transcription is documented as poor at every model size** — the project's own conclusion is that vanilla Whisper is the wrong tool entirely and a fine-tuned IndicWhisper/IndicVoices checkpoint would be needed. Real human voice has never been tested; the only clips that exist are Google TTS output, which is cleaner than a real phone microphone and does not exercise the noise/silence conditions where Whisper actually hallucinates.

6.4 Precise citizen location (GPS pin)

Files: `backend/routes_citizen_report.py :: _validate_pin()`, `intelligence/config.py` (`PIN_MAX_DISTANCE_KM = 5.0`, `ASSET_PIN_SNAP_M = 300.0`).

A citizen may supply a live GPS reading from their phone in addition to picking a village from the dropdown. It is parsed, checked for being finite and within valid lat/lon bounds, and stored in `precise_lat` / `precise_lon` — **the village-resolved `latitude` / `longitude` columns are never overwritten by it.** `intelligence/recompute.py :: _apply_precise_coords()` substitutes the precise pin for the village centroid at clustering/scoring time whenever both are present, so downstream work-group separation and road/facility matching use the citizen's actual position, not the village-wide fallback. `ASSET_PIN_SNAP_M` (300 m) governs how close a pin must be to a registered school/hospital to be snapped to that exact facility rather than treated as an unnamed pin — chosen because rural GPS accuracy on standard phones fluctuates 10–30 m and school compounds themselves span tens to hundreds of metres.

6.5 Specific-facility selection on the intake form

Files: `backend/routes_gazetteer.py :: /gazetteer/facilities`, `routes_citizen_report.py`.

For education/health reports, the intake form can offer a candidate list of real, named facilities near the chosen village (`SCHOOL_CANDIDATE_RADIUS_M = 2000.0` for schools, `HEALTH_NEAREST_MAX_KM = 8.0` for health, both matching the same radii the scoring engine uses elsewhere for consistency). If the citizen picks one and it is within `FACILITY_MAX_DISTANCE_KM = 10.0` km of the village centroid and matches the report's own category, `facility_id` is stored on the report. A facility later assigned by the demo-seeding script (not chosen by a real citizen) is honestly labelled `name_basis = "demo_assigned"` rather than `"citizen_selected"` when the asset is built.

6.6 DBSCAN clustering

File: `intelligence/clustering.py` (131 lines).

Reports are gated to the **same** `issue_category` **and within** `GATE_RADIUS_KM = 8.0` **km** before any semantic comparison happens — cheaper (small matrix) and more correct (two identical sentences about two different districts are not the same problem). The actual distance fed to DBSCAN combines geography and meaning:

```
distance(i, j) = haversine_km(i, j) + SEMANTIC_WEIGHT_KM(3.0) × (1 − cosine_similarity(i, j))
```

so two reports on top of each other geographically but saying completely different things ("the school roof leaks" vs "the borewell is dry") are pushed apart in the combined space. Pairs beyond the 8 km gate get an effectively-infinite penalty (`GATE_PENALTY_KM = 100 0.0`). `DBSCAN(eps=6.0, min_samples=3, metric="precomputed")` then runs separately per category — **8 km/6 km was tuned against the real 1,008-report dataset**, not guessed (measurement table reproduced in Part 9.3). Noise points (label `-1`) are deliberately **not** forced into a cluster: a single uncorroborated report is not yet a demand signal.

A second pass, `_attach_stragglers()`, then folds any *still*-unclustered report into an already-established cluster of the same category within the same gate radius. This is principled rather than a fudge: DBSCAN's `min_samples=3` rule is about *forming* a cluster from nothing, and once a problem is already established, a later nearby report is genuine additional evidence for it, not a new unverified claim.

6.7 Sentence embeddings

File: `intelligence/embeddings.py` (59 lines). `paraphrase-multilingual-MiniLM-L12-v2`, 384-dim, L2-normalised on encode so cosine similarity in `clustering.py` is just a dot product. Verified sanity check (quoted in `README.md`): "सड़क बहुत खराब है" vs "the road is very bad" → cosine 0.971; vs an unrelated water complaint → 0.105. Loaded once, lazily, and kept in memory rather than a vector database — SQLite has no pgvector equivalent anyway, and the build plan explicitly allows this for a dataset this size.

6.8 Population affected

File: `intelligence/population.py` (70 lines). Sums real Census `gazetteer.population` for every settlement within a **category-specific catchment radius** of the cluster/asset centroid — health 8 km (people travel furthest for a clinic), road 5 km (a broken segment strands a corridor), education 4 km, water 3 km (the most local of the four). Settlements with no recorded population contribute **nothing**, not an estimate — 948 of 1,042 gazetteer rows carry a real Census figure, and this is explicitly the single most load-bearing input to the score, so inventing the rest would be the worst place to guess.

6.9 The priority-score formula — every term, exactly as computed

File: `intelligence/scoring.py` (423 lines), constants in `intelligence/config.py`.

Stage 1 — Gap Score (0–1):

```
gap_score = 0.25·demand + 0.25·population + 0.25·infra_deficit + 0.25·vulnerability
```

Stage 2 — Priority Score (0–100, what the dashboard ranks on):

```
priority = gap_score × GAP_SCORE_MAX_POINTS × category_importance × confidence_gate
         + equity + strategic + urgency + feasibility − cost_penalty
```

Addition, never multiplication, by explicit design: multiplying five terms averaging 0.6 each collapses to 0.078, and one missing input would zero an otherwise-real gap. The **only** multiplier anywhere in the formula is the confidence gate, which damps and never zeroes.

⚠️ **Finding:** `GAP_SCORE_MAX_POINTS` **and** `URGENCY_POINTS` **are each assigned twice in** `config.py` (lines 108/121 vs 107/120) — `8 1.0` → `69.0` for the gap ceiling and `3.0` → `15.0` for urgency. Python keeps only the *last* assignment, so the values actually in force today are `GAP_SCORE_MAX_POINTS = 69.0` and `URGENCY_POINTS = 15.0` (matching the newer emergency-urgency redesign, 69+10+4+15+2=100), but the file still contains the old values as live, unshadowed-looking code above them, and `WEIGHTS.md`'s own

table lists both the old and new value on adjacent rows without marking the old one superseded. This is not a functional bug (the math is correct for anyone who reads to the bottom of the file), but it is a real piece of leftover code from the urgency-term rewrite that was never cleaned up, and it would mislead anyone skimming the top of the file or the corresponding `WEIGHTS.md` table row.

Term	Formula	Real data available?
`demand`	`min(1.0, unique_reporters / 25)` — capped, on purpose, so a well-connected area cannot out-shout a cut-off one by filing more complaints. "Unique reporters" = distinct signed-in accounts, falling back to distinct raw text for anonymous reports (`recomputed.reporter_key`)	mechanism only; real citizen identity still needed to fully close the "500 forwarded WhatsApp messages ≠ 500 corroborations" gap
`population`	`log(1+pop) / log(1+100,000)`, capped at 1.0 — log-scaled so a city and a hamlet compare on diminishing returns instead of raw headcount	✅ real (Census 2011)
`infra_deficit`	per-category, see 6.9.1 below	✅ real for road (Census + PMGSY + optional UDISE+ school override), water (JJM current coverage preferred over Census), health (measured against IPHS norms), education (RTE-violation signal); proxy (reported severity) only when no village nearby has any real record
`vulnerability`	multi-signal mean of power-hour deficit, no-drainage share, no-internet-CSC share, all Census 2011 area-level statistics — **never individual-level or caste data, by standing decision**	✅ real where any signal exists; proxy (settlement-size) otherwise
`confidence_gate`	`min(1.0, avg_confidence / 0.75)`, and multiplied by `NO_REAL_DATA_CONFIDENCE_FACTOR = 0.85` whenever infra or vulnerability had to fall back to a proxy	damps, never zeroes
`equity`	proportional to `shared_digitally_impaired` (BharatNet fibre status at nearby GPs) × 10 points when real BharatNet data exists; else a binary +10/+0 against a 10-block hardcoded list	✅ real (BharatNet 2022) in most catchments
`strategic`	+4 points if a published scheme rule is actually crossed (PMGSY population floor + no all-weather road / IPHS sub-centre entitlement / RTE middle-school breach / JJM sub-100% coverage), else a bare `population ≥ 5,000` proxy	✅ real per category, see 6.9.2
`urgency`	**emergency-only** since Feature #7 — see 6.10	✅ real signals combined, see below
`feasibility`	continuous linear fade from full credit at 15 km to zero at 45 km of the nearest real Census-recorded town, blended 70/30 with real all-weather-road-connectivity share	✅ real where Census `dist_nearest_town_km` exists; else falls back to a straight-line distance this project computes itself to an administrative HQ
`cost_penalty`	`-3.0 × min(1.0, population_affected / 50,000)`, subtracted	❌ no real per-project cost data behind this term at all — the team's own build plan calls this "the weakest-verified data category", and nothing since has changed that

The nine breakdown terms sum exactly to `priority_score`, verified by `test_breakdown_sums_to_score` and stated to have zero mismatches across all live clusters. This is what lets the dashboard answer "why this score" by literally printing its parts.

6.9.1 `infra_deficit`, per category (`intelligence/realdata.py`)

- **Road** (`_road_deficit`): the obvious "does it have an all-weather road" test was measured after loading and fails for only 12 of 942 villages — too flat to rank on, so the real signal leans on black-topped-vs-gravel surface share and, more strongly, on any **undelivered PMGSY work** near the cluster (a sanctioned-but-unbuilt road is treated as the government's own finding that the connection is missing — a saturating signal capped at 1.0). If the asset is a specific school, `school_condition_deficit()` (real 2024-25 UDISE+ classroom/electricity/water/toilet data) **overrides** the village-wide Census signal for that exact building, never blended with it.
- **Water** (`_water_deficit`): **current JJM tap-connection coverage is preferred over Census whenever it has been crawled**, because Census water data (2011) predates JJM (launched 2019) and measurably overstates need. Falls back to Census treated-tap / summer-failure shares only where JJM has not been crawled for that village.
- **Health** (`_health_deficit`): measured *against the IPHS population norms* (1 sub-centre per 5,000, 1 PHC per 30,000), not against other villages — "short of a published standard by this much" is something an official can act on; "worse than average" is not. Also folds in doctor-vacancy rate and share of villages more than 5 km from any facility.
- **Education** (`_education_deficit`): primary-school coverage is complete across all 942 villages in these districts, so the RTE signal necessarily comes from middle/secondary provision instead — secondary is weighted at 0.7× a middle-school gap, since only middle school is a statutory RTE duty.

6.9.2 `strategic` — real scheme eligibility, per category (`real_scheme_eligibility()`)

Road: PMGSY's own population floor (500 plain / 250 hilly-tribal) crossed while lacking an all-weather road. Health: population divided by the IPHS sub-centre norm exceeds existing sub-centre count. Education: RTE Act 2009's statutory middle-school-within-3km duty, proven only where a Census distance *band* (`b` =5-10km, `c` =>10km) actually confirms the breach — band `a` (<5km) is deliberately **not** counted as a violation even though a village at 4.9km could still be in breach, because Census only records a band, not an exact distance, and this project chooses to understate a deficit rather than invent proof it doesn't have. Water: any JJM-measured village below 100% household tap coverage.

6.10 Emergency Urgency (Feature #7) — the rebuilt urgency term

Files: `intelligence/emergency.py` (315 lines), `intelligence/config.py`.

Urgency is now **defined as an emergency, and only an emergency: bridge or building breakage, including cracks** — never for potholes, a missing teacher, a missing doctor, or a dry tap (those are handled by `demand`, `infra_deficit`, and the burst-detection/stale-record mechanism instead). Graded on a 3-point scale and scaled by an independent confidence in [0, 1]:

```
urgency_points = URGENCY_POINTS(15.0) × grade × confidence
grade  ∈ {crack: 1/3, partial: 2/3, collapse: 1.0}
```

`detect_report_emergency()` matches report text against multilingual (EN/HI/MR, Devanagari + transliterated) bridge/building keyword lists and three damage-tier keyword lists (collapse > partial > crack), with a road-specific guard: "sadak tut gayi" (road surface broken) is *not* counted as structural collapse unless a bridge/culvert word is also present, because road condition is `infra_deficit`'s job, not an emergency's.

`evaluate_emergency_signals()` combines **four independent signals, none decisive alone**:

Signal	Contribution cap	Source
Catastrophic wording, ≥2 distinct reporters	0.50	citizen text
Catastrophic wording, 1 reporter	0.40	citizen text
Report-burst velocity (`burst_ratio` / `velocity_term`)	0.30	timing of reports in this cluster
Active SACHET alert / CWC river warning nearby	0.20 (SACHET) + 0.15 (CWC), capped 0.30 combined	live government hazard feeds
Photo damage grade (C6 vision-LLM, or the pothole/crack model as a weak fallback)	0.40	`backend/photo_checks.py`

Confidences sum (capped at 1.0), with a further +0.20 bonus when two or more independent signal families agree something is broken. **If no signal fires at all, urgency is strictly 0.0 — never a guess.** A real photographed collapse graded by the vision-LLM is tracked separately (`collapse_confirmed_by_photo`) so the evidence panel can honestly distinguish photo-confirmed collapse from wording-only collapse.

⚠️ **Finding, confirmed by reading `intelligence/scoring.py`:** `score_cluster()` computes `urgency = urgency_points(issue_category)` and then **immediately overwrites** it on the very next line with the real emergency-signal call. Because `urgency_points()`'s own first check is `if isinstance(grade, str) or grade is None: return 0.0` (a legacy positional-call guard for exactly this shape of call), the first line always evaluates to `0.0` and is discarded before it can affect anything — it is dead code left over from the pre-Feature-#7 flat-urgency design, not a live bug, but it is a real, verifiable leftover that a future edit could trip over if someone assumes that first line still does something.

⚠️ **Second finding, confirmed by reading `intelligence/recompute.py :: _score_members()`:** `hazard_flag, hazard_evidence = realdata.hazard_near(...)` is called **twice** — once correctly scoped inside `if category in ("road", "water")`, and then unconditionally immediately afterward for every category, which **overwrites** the scoped result. The practical effect is that hazard/SACHET/CWC corroboration is actually computed and attached for **every** category (health and education included), not just road/water as the surrounding comment claims. This may well be the intended current behaviour (a flood alert plausibly matters for a health facility too), but as written it silently contradicts its own comment, and the first, category-scoped call is unreachable dead code exactly like the `urgency` case above — both read as remnants of the same incremental-rewrite pattern across this file's development history.

6.11 Stale-record trust-blend

File: `intelligence/scoring.py :: record_freshness()`, `intelligence/recompute.py :: resolve_infra_vintage_years()` / `resolve_vulnerability_vintage_years()`.

Government records lose trust exponentially with age, but **only when actively contradicted** by a real, corroborated burst of high-severity citizen reports:

```
freshness = 0.5 ^ (vintage_years / RECORD_TRUST_HALF_LIFE_YEARS(10.0))
contradiction = velocity_term × high_severity_share
trust = 1.0 − (1.0 − freshness) × contradiction
blended_value = trust × real_value + (1.0 − trust) × proxy_value
```

A 15-year-old Census record retains ~35% trust when *fully* contradicted; a record vintage of 0 (e.g. a live JJM crawl) is always treated as fully fresh. `resolve_infra_vintage_years()` resolves the record's actual age per source: UDISE+ 2024-25 school data → ~2 years old, JJM current coverage → ~2 years old, a PMGSY-sanctioned road's oldest undelivered sanction year → its real age, and Census 2011 for everything else → 15 years old as of 2026. The half-life (10 years) was an explicit owner decision recorded in `FEATURE_ROADMAP.md`, chosen because it restores roughly 10 of a real 20-point infra_deficit loss in a worked case study (the "Shinganapur" scenario) versus only ~6 points under a 20-year half-life.

6.12 Village → Asset priority view (Feature #2 — the dashboard's primary unit today)

Files: `intelligence/recompute.py :: _build_assets()` / `_build_villages()`, `backend/routes_villages.py`.

This is the newest and largest structural change since the old audit. Every citizen report is resolved down to a specific, named **asset**, by a five-rule cascade run inside `_build_assets()`:

1. **Citizen-selected facility** — if the report carried a `facility_id` matching its own category, use it (`name_basis = "citizen_selected"`, or `"demo_assigned"` if it was the synthetic-seeding script that attached it, never falsely labelled as a citizen's choice).
2. **GPS pin within `ASSET_PIN_SNAP_M` (300 m) of a facility** — snap to it (`"nearest_register"`).
3. **Health report with no pin** — nearest health facility within `HEALTH_NEAREST_MAX_KM` (8 km); if none, an honestly-labelled `"unresolved"` health bucket for that village.
4. **Education report with no facility and no pin** — an `"unresolved"` education bucket, with up to 6 real candidate schools listed for a human to choose from (`SCHOOL_CANDIDATE_RADIUS_M = 2000 m`, `SCHOOL_CANDIDATE_MAX = 6`).

5. **Road/water (or anything else)** — single-linkage grouped within `ASSET_GROUP_RADIUS_M` (250 m) per village, then named: a GeoSadak segment within 500 m if the report has a real pin (never for a village-centroid-only report — a road that merely passes near the village centre is not "the" road being complained about), else the single sanctioned PMGSY work for that village if unambiguous, else a generic "Road near {village}" with the real candidate works listed.

Each asset gets its own full `score_cluster()` run (not a rollup of its parent cluster's score — an asset's catchment population, real-data lookups and evidence are computed fresh around its own coordinates). `_build_villages()` then rolls every village's assets into one `VillagePriority` row: **the village's number is always its single worst-off asset's score** (a standing, explicit decision — never an average or sum across categories), plus the independent `has_urgent_asset` flag described in Part 5.3.

`GET /villages/{gazetteer_id}` (`routes_villages.py`) returns the full village panel: every asset ranked, the top asset's complete breakdown/evidence, and the village's citizen reports — with a privacy guard (`_format_coord()`) that rounds any non-register (i.e. potentially a real citizen's exact GPS pin) coordinate to `ASSET_PUBLIC_COORD_DECIMALS = 3` decimal places (~110 m at these latitudes) before it ever leaves the server, while a register-sourced facility (public PMGSY/UDISE data, not personal) keeps full public precision.

6.13 Real road geometry — PMGSY GeoSadak (Feature #3)

Files: `backend/load_pmgsy_geosadak.py`, `intelligence/realdata.py :: nearest_road_segment()`.

Physical line geometry (not just point coordinates) for real, officially-named roads, sourced from `github.com/datameet/pmgsy-geosadak`'s mirror of the Ministry of Rural Development's Road_DRRP layer (`data/Road_DRRP/Maharashtra.zip`, originally under the Government Open Data License). Matching distance was **tuned live, not guessed**: at the original 150 m radius, 84 of 199 real road assets (42%) never matched any segment; 250 m recovered 31 of those, 400 m recovered 64, 600 m recovered 81, 1000 m recovered all 84 — but a village can have more than one nearby road, and too generous a radius risks confidently naming the *wrong* one, which is worse than the honest "Road near {village}" fallback. **500 m was chosen as the documented middle ground.** Point-to-polyline distance is computed with an equirectangular projection (accurate to centimetres at this local scale) after a cheap bounding-box pre-filter, since the segment table holds thousands of rows checked once per road asset.

6.14 Demand-vs-Investment alignment

Files: `intelligence/investment.py` (354 lines), `backend/routes_intelligence.py`.

Answers the research thesis's "Capability E" question directly: is citizen demand pointed at places the government has already funded, or unfunded, or is money sitting near places nobody is complaining about? Two structurally different implementations exist side by side, because "funded" means a genuinely different thing in each domain:

- **Villages vs. roads** (`build_village_investment`): joins gazetteer villages, **road-only** citizen reports within 5 km, and PMGSY works pinned to that village. Classifies each into `funded_undelivered` (a sanctioned, still-undelivered work + real nearby demand), `demanded_unfunded` (real demand, no sanctioned work found), or `funded_not_demanded` (a sanctioned work, little/no demand — explicitly the weakest of the three while citizen demand is still synthetic, and labelled as a mechanism demonstration, never a finding). ⚠️ **This was a confirmed, fixed bug**: the counting used to include health/school/water complaints against road-only PMGSY works, inflating `demanded_unfunded`; the code now filters `issue_category == "road"` before counting (verified in the current source, dated fixed 2026-09-15 per `FEATURE_ROADMAP.md`).
- **Schools** (`build_school_investment`): a genuinely different "funded" — real UDISE+ `total_grant` vs `total_expenditure` for a *specific school this exact year*, crossed with whether that school also carries a real, currently-recorded physical deficiency (`school_condition_deficit`). Kept in its own `SchoolInvestment` type rather than merged into the village buckets, because a PMGSY work-status string and an UDISE+ grant balance answer different questions and conflating them would misrepresent both.

Only ~33% of PMGSY works pin to a known village (1,042-row gazetteer vs. thousands of real habitations in these districts); unpinned works are counted and reported separately, never silently dropped.

6.15 Funding warnings

File: `backend/routes_dashboard.py :: _funding_warnings()`.

Turns evidence recompute.py already computed into an explicit, human-readable caution or confirmation on the cluster/asset detail panel itself, for all three categories with real funding data (road via PMGSY, water via JJM per-scheme cost/status, education via UDISE+

grant/expenditure): `"already_funded"` (money already committed, check delivery before recommending more) or, the newer, previously-silent case, `"unfunded_need"` (a real, documented deficiency exists with *no* funding record at all in the one dataset that would show it — surfaced explicitly rather than looking identical to "no evidence either way").

6.15a Photo preview in officials' report lists

Files: `backend/routes_citizen_report.py :: attach_photo_info()`, `frontend/js/app.js :: renderReportsList()`.

The cluster dock, village panel and asset panel all share one report-list renderer; each report row that has a real attached image gets a "📷 Image (N)" button that reveals the photo(s) inline on click rather than always rendering them (most reports have none). `attach_photo_info()` batches one query across a whole reports list rather than one per report, so a list with hundreds of rows stays cheap regardless of which of the three panels calls it.

6.16 Budget optimizer — real constrained allocation

File: `intelligence/budget_optimizer.py` (166 lines), `backend/routes_intelligence.py :: GET /budget-optimizer`.

Answers a genuinely different question from the old flat what-if re-scoring (now removed): "given exactly this much money, which specific real projects should be funded to reach the most people without exceeding budget" — an exact **0/1 knapsack**, not a greedy approximation or a re-rank. Scoped deliberately to **road and water only**, because health and education have no honest per-project *cost to fix* in this database (UDISE+ grants are "money already given", not "cost of the specific fix needed"). Cost is never re-derived — it's reused directly from each asset's own already-computed evidence (`undelivered_sanctioned_cost_lakh` for roads, `unspent_estimate_lakh` for water schemes with at least one undelivered scheme), so the optimizer stays consistent with what the dashboard already shows for that exact asset. An asset with no real recorded cost is excluded from the optimizer entirely rather than given a guessed cost — it stays visible in the ordinary ranking, just outside this feature's scope. The DP works in whole-lakh precision units (`COST_PRECISION_LAKH = 1.0`), which at a few hundred real candidates is trivially small for exact dynamic programming, and can optimise for either total population reached or priority-weighted benefit (`value="population"|"priority"`).

6.17 Fresher/live government data, evidence-only (`intelligence/realdata.py`)

All of the following feed a cluster/asset's `evidence` block but **never move a score term** — a deliberate, repeated caution across every one of them in the source comments, because their shape had not yet been checked against live clusters when they were first wired:

- **Groundwater** (`lookup_groundwater`, NWDP telemetry, ≤25 km) — real depth-below-ground and trend at the nearest DWLR station, corroborating a water-scarcity complaint against an objective hydrological signal rather than only what the citizen said.
- **JJM water testing** (`water_testing_catchment_evidence`) — how many of a catchment's villages were actually sampled this financial year and how many samples were taken; this is **testing coverage**, not contamination results (the `Contaminantwise` report on the same site was never actually tried).
- **JJM per-scheme cost/status** (`jjm_scheme_catchment_evidence`) — the Village → Scheme → Cost → Status chain for real water-supply works, distinct from the tap-coverage percentage used in `infra_deficit`.
- **MGNREGA expenditure** (`mgnrega_catchment_evidence`) — real rural-employment-guarantee spending (wages/material split) near a road or water asset, scoped to those two categories since that is what MGNREGA public-works spending is actually about.
- **Historical flood exposure** (`flood_exposure_evidence`, NDEM 2013/2021 satellite extents) — applies to **every** category (a flood affects whatever is standing in it), a bounding -box lower-bound distance check that recovered a real performance regression when first wired in (a full recompute went from well-under-a-minute to over two minutes checking 14,434 events per cluster/asset; fixed with a cheap plain-degree pre-filter before the real haversine check, restoring it to ~52 seconds).
- **Live hazard corroboration** (`hazard_near`) — an active SACHET disaster alert or an unusually recent CWC river reading within 25 km/72 hours. **Known, documented weakness**: it flags "hazard" whenever *any* reading exists nearby in the window, without checking whether the value itself is unusual — a calm river at normal level is treated the same as a flood level. Acceptable for an evidence-only signal today; would need a real threshold before being allowed to move a score.
- **District-level GPDP investment context** (`gpdp_district_evidence`) — real panchayat plan counts, approved activities and estimated outlay per district, deliberately kept district-level only and never attributed to a specific village or asset, because a district total cannot be honestly split down.
- **Village-panchayat finance** (`village_finance_evidence`, PRIASoft) — real tied/untied 15th Finance Commission receipts and payments for the whole panchayat (not scheme-specific like the road/water numbers above), giving every village its own real

unspent-grant balance independent of PMGSY.

6.18 Photo verification and fake-complaint defence (Workstream C)

Files: `backend/photo_checks.py` (1,027 lines), `backend/routes_photo_checks.py` (338 lines), `frontend/js/camera-capture.js`, `frontend/js/photo-checks.js`, `frontend/photo-lab.html`.

Governing rule, stated in the module's own docstring and enforced throughout: no check ever rejects a complaint. Each one only raises or lowers a report's confidence (never below half the original) and, for the stronger signals, adds the report to an officer's review queue.

Check	Mechanism	Real limitation, stated honestly
C1 — capture provenance	`getUserMedia` /ImageCapture API captures GPS + timestamp *at the shutter press*, not a later map pin; flags a photo taken >5 km from the chosen village (`CAPTURE_MAX_DISTANCE_KM`), captured >120 min before upload, uploaded (not live-captured), or missing GPS entirely	Client-supplied metadata can be forged by a determined attacker; this raises the cost of faking a photo, it is not proof
C2 — distinct accounts, not wording	`recompute.reporter_key()` : one signed-in account = one voice; falls back to distinct raw text only for anonymous reports	Real corroboration counting still needs universal citizen identity
C3 — duplicate photo	Perceptual hash (`imagehash.phash` / `dhash` , Hamming distance ≤8/≤12) against every stored photo; flags reuse by a different account or village as `reused_photo` (strong penalty), same account+village as `same_photo_resubmitted` (no penalty — legitimately re-attaching evidence)	—
C4 — edit detection	Editing-software name in EXIF metadata → real, weighted flag. Error-level analysis (re-save at known JPEG quality, find a texture-adjusted outlier region) is shown as an **advisory heat map only** — on a fresh test sample it flagged 12% of genuine photos and 12% of spliced ones, i.e. **no discrimination at all**, and its flag weight is explicitly set to `0.0` in `FLAG_WEIGHTS` so it costs an honest citizen nothing	Could not separate edited from genuine photos in testing; kept only as a visual aid for an officer
C5 — road defect detection	The team's own YOLOv8 model (`models/pothole_best.pt` , 25.9M params, classes `crack` / `pothole` , run at native 1024 px with test-time augmentation) feeds `evidence["photo_defect"]` on road assets — road *condition* corroboration, not urgency	Measured accuracy on 88 held-out real photos: 81.8% accuracy, 74% precision, 86% recall, F1 0.80, ROC-AUC 0.92 (`models/README.md`); misses water/debris-filled potholes, false-alarms mostly on dirt roads
C6 — building/bridge damage grade	Grades crack < partial < collapse from **both** photo and text. The photo signal is a real, general-purpose **vision-language model** (default `antigravity/gemini-3.7-flash-medium` , called through a local OmniRoute gateway subprocess), because no trained ground-level bridge/building collapse classifier exists anywhere to install — a coarse scene judgment ("does this look collapsed") is exactly what a general VLM is good at and a narrow detector structurally cannot do. Falls back to the pothole/crack model's confidence as a weak proxy signal (lower weight, 0.30 vs 0.55) only when the VLM call fails or is disabled	Explicitly prototype-stage: routed through a dev-machine gateway convenience, not a direct provider key; a failure (timeout, bad JSON, server down) returns `None` and silently falls back to wording-only grading rather than fabricating a grade
C7 — screen replay	Moiré-pattern spectral analysis (isolated FFT peaks 6σ above a local median, in colour-opponent channels where a screen's RGB sub-pixel grid shows up strongest) **plus** a 5-frame burst liveness check (mean frame difference; a still image fed through a virtual camera shows near-zero difference and no sensor noise, a real handheld camera always has both)	Calibrated on 70 genuine photos (none scored above 0.3) against simulated screen recaptures (69% flagged) — not tested against a real screen photographed with a real phone yet

Combined authenticity is multiplicative across fired flags (`authenticity = ∏(1 − FLAG_WEIGHTS[flag])`, floored at `AUTHENTICITY_FLOOR = 0.2`), and `confidence_adjustment()` damps a report's overall `confidence` toward `confidence × (0.7 + 0.3 × worst_authenticity)`, or **lifts** it (toward `+25%` of the remaining gap to 1.0) when a live, road-relevant photo genuinely shows the claimed defect at or above threshold. **Never below half the original confidence, and never above 1.0** — exactly the same "damp, never zero" rule the priority-score confidence gate follows.

6.19 Dashboard

Files: `frontend/index.html`, `frontend/js/app.js` (1,996 lines), `frontend/js/panels.js` (584 lines).

A D3.js SVG map with a single fitted Mercator projection and one `<g id="viewport">` that every zoom level (India → Maharashtra → District → Village/Cluster → Asset) transforms affinely rather than re-projecting geometry per frame; stroke widths, bubble radii and pin icons are continuously divided by the live zoom factor so they stay legible at every scale. District level switches between a **Villages mode** (bubbles by village, sized by report count, coloured by priority + an urgency indicator) and the older **Clusters mode**. Clicking down to a village opens the village panel and draws its assets, including real PMGSY road geometry where available, de-stacking overlapping pins with a golden-angle radial spread. A bottom "dock" opens with a three-column breakdown: the 9-term score with evidence provenance per term, the underlying government-record facts (with a clear real-vs-proxy label per term), and the anonymised citizen reports behind it, including any attached photo evidence via the photo-checks panel. Verified real fetch calls (grepped from the actual source, not assumed): `/clusters`, `/clusters/{id}`, `/clusters/{id}/reports`, `/villages`, `/villages/{gazetteer_id}`, `/assets/{id}`, `/map-data`.

6.20 Citizen authentication and tracking

File: `backend/routes_auth.py` (303 lines). PBKDF2-SHA256 (100,000 iterations, random 16-byte salt), `secrets.compare_digest()` for constant-time verification, opaque `secrets.token_urlsafe(32)` bearer tokens stored in `user_session` (no expiry currently enforced — the column exists, nothing writes to it). `GET /citizen-report/{id}` and `GET /citizen/my-reports` expose three honest states: `unresolved` (location never matched), `awaiting_corroboration` (resolved but not yet in a cluster — the normal state for a fresh report, not an error), or `clustered` (with a live rank).

⚠️ **Confirmed live bug, present today:** `get_my_reports()` in `routes_auth.py` references `cl.title`, `latest_score.rank`, and `latest_score.final_score` — **none of these columns exist** on `DemandCluster` or `PriorityScore` (the real column names are `priority_score` and there is no `rank` / `title` / `final_score` field anywhere in `models.py`). This is the exact same class of bug flagged in an earlier version of this project's own audit years of commits ago, and it has evidently never been fixed in this specific function even as the surrounding codebase has been rewritten many times since — `GET /citizen/my-reports` will raise an `AttributeError` at runtime the moment a signed-in citizen with a clustered report calls it.

PART 7 — Complete data-source inventory (every `load_*.py` script)

Every URL below was read directly out of the current source file, not summarised.

Loader	Table(s) written	Real source	Method / honesty notes
`load_gazetteer.py`	`gazetteer`	OpenStreetMap Overpass API (`overpass-api.de/api/interpreter`)	`place=village|town` nodes per district; falls back to a small hand-curated real-place list if Overpass fails/rate-limits
`load_demographic_data.py`	`gazetteer.population`	Census 2011 District Census Handbook, via a GitHub CSV mirror (`github.com/bnamita/Village_Mapping_v2`) of the same data `data.gov.in` serves as resource `2e03171c-…` — the official API's public demo key was tested and rejected ("Key not authorised"), so this mirror avoids a personal registration step at the cost of trusting a third party instead of the government API directly	Fuzzy name match, `rapidfuzz`
`load_village_amenities.py`	`village_amenities`	Same Census CSVs as above, 280+ columns (road/water/health/school/connectivity/isolation)	SC/ST population columns deliberately not loaded
`repair_data.py`	`gazetteer` (block, taluka HQs), `citizen_request` (coord backfill)	Hand-entered real taluka HQ coordinates (27 talukas)	Fixes the missing "Karvir" taluka and the historic `lat` / `latitude` key-mismatch bug that once left every report's coordinates null
`load_lgd_hierarchy.py`	`lgd_village`	Local Government Directory, via `github.com/ramSeraph/opendata` 's daily 7z-compressed mirror of LGD's own bulk exports	Needs `py7zr` ; resolves the newest release filename at runtime rather than pinning a date
`load_bharatnet.py`	`village_amenities.conn_bharatnet_*`	BharatNet/BBNL, `storage.googleapis.com/bbnl_data/parsed.zip` , public no-login	Spatial match (nearest GP fibre node), not name match — a GP covers several villages
`load_udise_schools.py`	`public_facility` (schools)	UDISE+ school directory, via DataMeet's GitHub GeoJSON mirror (`github.com/datameet/udise_schools`)	Chosen over OSM (2 schools found within 6 km of a test village), over the UDISE+ SPA (no documented API), and over data.gov.in (aggregates only, not a directory)
`load_health_facilities.py`	`public_facility` (health)	NIC HealthGIS, via a GitHub release mirror (`github.com/yashveeeeeeer/india-geodata`), originally India Open Government Licence	147,957 features nationally, filtered to the two pilot districts
`load_pmgsy_works.py`	`government_project`	PMGSY's live dashboard (`pmgsy.dord.gov.in/dbweb/ChiefSecretary/GetRoadTenderDetails`), public, session-cookie + anti-forgery token obtained first	Only returns tender/execution-pipeline works, never "Completed" — a backlog, not the full programme, never summed against the district aggregate
`load_pmgsy_geosadak.py`	`pmgsy_road_segment`	PMGSY GeoSadak Road_DRRP layer, via DataMeet's mirror (`github.com/datameet/pmgsy-geos`	Real line geometry; no surface/condition attributes in this layer

Loader	Table(s) written	Real source	Method / honesty notes
		`adak`), Government Open Data Licence	
`load_jjm_water.py`	`village_amenities.jjm_*`	JJM Citizen Corner's real JSON endpoint (`ejalshakti.gov.in/jjm/citizen_corner/VillageInformation.aspx/BindHabitationInfo`)	No bulk export exists — a rate-limited, disk-cached crawler discovers village IDs through the page's own ASP.NET postback chain
`load_jjm_water_testing.py`	`water_testing`	JJM WQMIS (`ejalshakti.gov.in/WQMIS`)	The site encrypts every dropdown value with a **fixed, hardcoded AES-128 key found in its own JavaScript** (`8080808080808080`, used as both key and IV) — not real security, an obstacle this loader replicates
`load_jjm_village_schemes.py`	`jjm_village_scheme`	ejalshakti.gov.in's village-profile report (`JJM/JJMReports/profiles/rpt_VillageProfile.aspx`) — a genuinely different report from the Citizen Corner above	Requires the LGD hierarchy loaded first (looked up by LGD village code); deliberately omits O&M staff/committee member names present on the same page
`load_mgnrega_expenditure.py`	`village_mgnrega_expenditure`	MoRD's NREGA Soft portal (`mnregaweb2.dord.gov.in/netnrega/state_html/gp_cummulative_report1.aspx`)	Uses hardcoded, pre-captured ASP.NET postback payload tokens per district — a real fragility: these can expire/rotate server-side and silently break the loader with no warning if the site's session mechanism changes
`load_priasoft_receipt_expenditure.py`	`village_panchayat_finance`	eGramSwaraj/PRIASoft (`egramswaraj.gov.in/recExpVpNew.do`), Ministry of Panchayati Raj, 15th Finance Commission scheme code 3287	No CAPTCHA; verified live: 4,826 rows, Kolhapur 66.6% village-matched, Nashik 23.5% (genuinely explained by Nashik's gazetteer having far fewer villages than PRIASoft's habitation list, not a bug)
`load_gpdp_district_summary.py`	`gpdp_district_summary`	eGramSwaraj public dashboard (`egramswaraj.gov.in/index.do`)	District-level totals only; deliberately never wired into any village/asset score
`load_cwc_river_levels.py`	`river_reading`	Central Water Commission flood-forecast API (`ffs.india-water.gov.in/iam/api`)	Live station search + most-recent-reading lookup
`load_sachet_alerts.py`	`hazard_alert`	NDMA's SACHET public CAP alert feed (`sachet.ndma.gov.in/cap_public_website/rss/rss_maharashtra.xml`)	Standard CAP 1.2 XML, parsed for event/severity/urgency/effective/expires/districts
`load_nwdp_groundwater.py`	`nwdp_groundwater`	National Water Data Portal (`nwdp.nwic.gov.in`), verified open unauthenticated CSV telemetry resources	Also writes a local CSV snapshot (`backend/data/nwdp_groundwater_stations.csv`) as a fallback read path
`load_flood_inundation.py`	`flood_event`	NDEM satellite-derived 2013/2021 flood-inundation extents, via `github.com/ramSeraph/india_natural_disasters`'s no-login parquet mirror of the same underlying government	Stores each event's real bounding box, not its exact polygon (no `shapely` dependency needed); a conservative, honest lower bound on true distance

Loader	Table(s) written	Real source	Method / honesty notes
		data Bhuvan's own (timed-out) WMS layer serves	
`load_udise_school_data.py`	`school_condition`	UDISE+ Know Your School's own JSON API (`kys.udiseplus.gov.in/web-app/api/school/report-card` and `/facility`)	Confirmed to return real JSON directly with an honest civic-research User-Agent, no login needed
`load_mosdac_rainfall.py`	`mosdac_rainfall` (0 rows, by design)	ISRO MOSDAC GSMaP rain product	**Confirmed dead end**: the bulk open-data endpoint 302-redirects to an ISRO Keycloak SSO login; the script checks this live, prints the honest verdict, and writes **zero** synthetic rows rather than fabricate anything
`capture_gpdp.py`	`village_budget_plan` (0 rows — never actually run)	eGramSwaraj GPDP report (`egramswaraj.gov.in/getGPDPReport.do`), CAPTCHA-protected	A semi-automated Playwright tool requiring a human to solve the CAPTCHA and press Enter per village; built but never executed at scale, and its own required Step 0 (test whether one CAPTCHA solve covers many villages) was skipped, so it silently assumes the expensive worst case (441 solves)
`seed_synthetic_data.py`	`citizen_request` (1,000 rows, `is_synthetic=True`)	Not a government source — generated demo data	Village selection weighted by real Census population; every complaint template runs through the actual `process_report()` pipeline (not hand-assigned fields), so it doubles as a load test; covers exactly the 4 categories the pipeline supports (an earlier version generated 2 unsupported categories and silently produced `issue_category=NULL` rows for a third of the batch — fixed)
`assign_synthetic_assets.py`	`citizen_request` (precise_lat/lon, facility_id, pin_source on synthetic rows only)	N/A — deterministic demo placement	Never touches a real (non-synthetic) report; `--undo` reverses it

Honest summary of what's real vs. not, restated plainly: of the nine priority-score terms, `population` , `infra_deficit` (road/water/health/education), `vulnerability` , `equity` , and `strategic` now run on real government records wherever a nearby village has any (with an honest, confidence-penalised proxy fallback where none exists). `unique_reporters` / `demand` and `cost_penalty` remain proxies with no realistic near-term fix (the first needs universal citizen identity this project doesn't have; the second needs real per-project cost data across health/education that this research explicitly could not find published anywhere). `mosdac_rainfall` and `village_budget_plan` are real tables that are correctly empty rather than fabricated.

PART 8 — Algorithms, precisely

1. **Haversine distance** (`intelligence/clustering.py` , `realdata.py` , `population.py` , `photo_checks.py` , `investment.py` — reimplemented independently in each file rather than shared, a minor duplication worth noting): standard great-circle

formula, `R = 6371.0` / `6371.0088` km (two slightly different Earth-radius constants exist between files — not materially different in practice, but a genuine inconsistency).

2. **DBSCAN over a combined geo+semantic distance matrix** — see Part 6.6.
3. **Gate/eps tuning table** (`intelligence/config.py` , measured on the real 1,008-report dataset spread over 439 villages):

gate	eps	% clustered	clusters	largest cluster
2 km	2 km	25%	68	11 — too sparse to demo
5 km	4 km	44%	113	11
8 km	**6 km**	**66%**	**113**	**37 ← chosen**
12 km	8 km	79%	75	101 ← over-merging

4. **Cosine similarity** on L2-normalised MiniLM vectors — a plain dot product (`vectors @ vectors.T`), clipped to [-1, 1] for float-rounding safety.
5. **RapidFuzz** `WRatio` fuzzy string matching — used for citizen-text geocoding (threshold 75/100) and for **every** government-data loader's village-name join (typically threshold 85–88), since almost none of the 15+ real data sources share a clean join key with the 1,042-row gazetteer.
6. **Single-linkage agglomerative grouping by radius** (`_group_by_radius()` in `recompute.py`) — used for both work groups (250 m) and the newer asset grouping (also 250 m); $O(n^2)$ but cluster sizes are small enough (single digits to a few dozen) that this costs nothing and avoids tuning a second DBSCAN pass.
7. **0/1 knapsack, exact dynamic programming** — `intelligence/budget_optimizer.py` , whole -lakh precision, trivially small at real candidate counts (a few hundred road/water assets).
8. **Error-level analysis (ELA)** — re-save at JPEG quality 90, compare each 16×16-pixel block's error against a texture-predicted baseline (robust linear fit, worst 20% of residuals dropped and refit), flag the **largest connected** region of statistical outliers rather than any isolated block (a pasted/retouched region is contiguous; noise is not). Explicitly downgraded to advisory-only after measurement showed no real discriminative power.
9. **Moiré detection** — 2D FFT of the colour-opponent channels (red−green, blue−green, luma) of a centred, windowed crop; isolated spectral peaks more than 6σ above an 11×11 median-filtered local baseline, with sensor axes and the JPEG 8×8 lattice explicitly masked out so they can't be mistaken for a screen's sub-pixel grid.
10. **Burst liveness** — mean absolute pixel difference between consecutive downsampled burst frames, plus a phase-correlation shift estimate; a real handheld camera always shows both non-zero difference (sensor noise) and a little motion.
11. **Perceptual hashing** (`imagehash.phash` / `dhash`) with Hamming-distance thresholds (≤8 phash, ≤12 dhash) for duplicate-photo detection, robust to resize/recompression.

PART 9 — Security review

Largely unchanged in posture from the project's earliest version, with new surface area from photo verification and the VLM gateway call:

Area	Status
Password storage	✅ PBKDF2-SHA256, 100,000 iterations, random 16-byte salt, constant-time compare
Session tokens	✅ `secrets.token_urlsafe(32)`, opaque, but **no expiry is ever enforced** — `UserSession.expires_at` exists and is never set by any code path
File upload path safety	✅ stored filenames are a random 32-hex token + an extension derived from the *declared* content-type, never the uploader's filename — path traversal via filename is not possible
File upload type/size limits	✅ whitelist (JPEG/PNG/WebP/HEIC/PDF), 4 MB/file, 5 files/report, matching CPGRAMS's own limits
PII segregation	✅ enforced by code structure — `citizen_identity` is imported by zero dashboard/analytics route files, not by a runtime access check
SQL injection	✅ SQLAlchemy ORM throughout, no raw string interpolation into SQL found in any reviewed file
CORS	⚠️ `allow_origins=["*"]` — fine for a prototype, must be restricted before any real deployment
Rate limiting	❌ none on any POST endpoint, including `/citizen-report` and the expensive `/recompute-scores`
`/recompute-scores` authorization	❌ none — anyone who can reach the API can trigger a full recompute (idempotent, but a resource-exhaustion vector at real scale)
Report/attachment IDs	⚠️ sequential integers, guessable; `/citizen-report/{id}` and `/report-attachment/{id}` are both unauthenticated by design (the citizen-tracking flow needs no login), so anything they return should be treated as effectively public — which is exactly why capture GPS coordinates and citizen identity are never included in their responses
Coordinate privacy	✅ `_format_coord()` rounds any non-register (potentially personal GPS) coordinate to ~110 m before it leaves the server; register-sourced (public PMGSY/UDISE) coordinates keep full precision, correctly, since those aren't personal data
Photo capture coordinates	✅ stored (`photo_check.capture_lat/lon`) but `public_view()` explicitly strips them before any response leaves the server — only a distance-to-village is ever returned
Third-party AES key in `load_jjm_water_testing.py`	The key (`8080808080808080`) is the *website's own* hardcoded obfuscation key, replicated to talk to a public government portal in its own expected format — not a secret this project owns or needs to protect, but worth knowing it's embedded in plaintext in a checked-in loader script
Hardcoded ASP.NET postback tokens in `load_mgnrega_expenditure.py`	A real operational fragility, not a security hole: these are long, pre-captured, opaque session/viewstate-style strings per district that will silently stop working if the government site's session handling changes, with no built-in detection of that failure mode beyond an empty result
Vision-LLM gateway dependency (C6)	The damage-grading call shells out to a local CLI (`omniroute api chat ...`) via `subprocess.run(shell=True)` with a JSON body written to a temp file — explicitly flagged in the code as a dev-machine convenience to swap out before real deployment; a failure of any kind returns `None` and falls back to wording-only grading rather than raising, so it cannot crash report submission

PART 10 — Confirmed bugs and code-quality findings from this audit

These were found by reading the actual current source, not inherited from any older document (though #1 below is the same *class* of error an earlier audit of this project once found in a different function, and it has resurfaced here uncorrected):

1. ✅ **Fixed, same day.** `GET /citizen/my-reports` used to crash for any signed-in citizen with a clustered report — `backend/routes_auth.py :: get_my_reports()` read `cl.title`, `latest_score.rank`, and `latest_score.final_score`, none of which exist on `DemandCluster` or `PriorityScore`. Fixed: `issue_category` (the real field every other cluster view already labels a cluster with) replaces `title`; `priority_rank` is now computed the same way `GET /clusters` itself ranks clusters (sorted by `priority_score` descending, via the existing `routes_dashboard._latest_score_by_cluster()` helper), so a citizen's own rank always matches what an official sees; `priority_score` reads the real column name. Verified live, twice — once against a fresh process (200 OK, correct data: cluster 42, rank 72, matching `/clusters`' own ranking exactly), and once by reproducing the original crash against an already-running server that still had the pre-fix code loaded, then restarting it. A new regression test (`test_my_reports_endpoint` in `test_citizen_report_endpoint.py`) exercises the exact crash scenario against the real database.
2. **Shadowed constants in `intelligence/config.py`**: `GAP_SCORE_MAX_POINTS` is assigned `81.0` then `69.0`; `URGENCY_POINTS` is assigned `3.0` then `15.0`. Python keeps only the final value (`69.0` / `15.0`, correctly matching the emergency-urgency redesign), but both old values remain as live-looking code above the real ones, and `intelligence/WEIGHTS.md`'s own table lists both without marking the first superseded — genuinely confusing for anyone reading top-to-bottom.
3. **Dead `urgency` assignment in `intelligence/scoring.py :: score_cluster()`**: the line `urgency = urgency_points(issue_category)` always evaluates to `0.0` (a legacy positional-call guard in `urgency_points()` catches it) and is immediately overwritten by the real emergency-signal call on the next line — harmless, but leftover from before Feature #7.
4. **Hazard-corroboration scoping bug in `intelligence/recompute.py :: _score_members()`**: `hazard_near()` is called once correctly scoped to `category in ("road", "water")`, then called again unconditionally immediately after, silently overwriting the scoped result — so SACHET/CWC hazard corroboration is actually attached to every category today, not just road/water as the surrounding comment states. Likely benign or even desirable in effect, but the code contradicts its own comment.
5. **Duplicate/redundant import** in `intelligence/recompute.py`: `from .scoring import score_cluster` immediately followed by `from .scoring import score_cluster, velocity_term` — harmless (Python re-binds the same names) but a sign the file has been incrementally patched by multiple passes without a final cleanup.
6. **Dead code**: `intelligence/realdata.py :: school_condition_evidence()` builds a real, human-readable sentence from UDISE+ data and is never called anywhere in `recompute.py` — only its sibling `school_condition_deficit()` (the numeric score) is used. Confirmed by `features.md` itself and independently verified here by grep.
7. **Inconsistent Earth-radius constant** across files that each reimplement haversine independently (`6371.0` vs `6371.0088` km in different modules) — a sub-100-metre-scale discrepancy, immaterial to any actual ranking decision, but a real duplication that a shared utility function would have prevented.
8. **No test file appears to cover** `get_my_reports()`, `hazard_near`'s category scoping, or the config.py shadowed-constant behaviour directly — the project's own test suite (291 intelligence tests, 66 backend endpoint tests) is large and was not exhaustively cross-referenced line-by-line against every function in this audit, so absence of a finding elsewhere in this document is not proof of absence of further issues.

None of the above were fabricated or inferred from documentation alone — each is a specific line or pair of lines in the current checked-in source, quoted or paraphrased with its exact location above.

PART 11 — Testing, as the project itself reports it

From `README.md`, dated 2026-09-16 (not independently re-run as part of this audit — stated here as the project's own current claim, distinct from the direct-source-reading findings above):

```
python -m intelligence.test_intelligence   # 291 checks (scoring, assets, C2 distinct accounts, real-data evidence)
cd pipeline && python test_pipeline.py     # 46 end-to-end cases
python test_photo_checks.py                # 65 checks for C1–C7, including the vision-LLM damage model
python test_citizen_report_endpoint.py     # 66 checks — writes to hackathon.db, back it up first
python test_assign_synthetic_assets.py     # 4 checks
```

Historically, this project has twice found and fixed test suites that ran and "passed" while counting for nothing — three pipeline test files that made zero assertions (fixed, now 34 real checks), and three P3 test functions bolted onto the bottom of `test_intelligence.py` as bare module calls that ran but were never in the counted total and could not have failed the exit code even if they broke (fixed, moved into `main()`'s list). This is exactly the kind of thing worth re-checking periodically rather than trusting a test count at face value forever — the project's own history is the best argument for that caution, not a criticism invented for this audit.

PART 12 — What's designed but not built, and what's explicitly ruled out

Condensed from `FEATURE_ROADMAP.md` and the more current `features.md` (which corrects several stale entries in the former — always prefer `features.md` where the two disagree, per the project's own instruction).

Designed, real research done, not yet coded: - **Silent Need Detector** (a real, non-flat equity signal): researched twice, including a live-verified literacy data source (`censusindia.gov.in` per-district Primary Census Abstract), but the owner's final decision was to leave the equity term as the flat BharatNet proxy — not because the idea is wrong, but because this project's own citizen report volume (67 of 2,014 reports carry a real citizen GPS pin; the rest are synthetic or bulk test data) is not enough to calibrate an expected-vs-actual reporting baseline honestly. - **Corroboration softening for 1–2-reporter assets** — discussed, never designed in detail, confirmed zero code by grep. - **Village-score aggregation redesign** (something more nuanced than "max of assets" per category) — real prior research exists (OECD/Alkire-Foster/NITI-Aayog/IMD/FEMA precedent was gathered) but the formula itself is unchanged; only the independent urgency flag was shipped as a stopgap. - **Feasibility Phase 2** (live OSM nearest-town distance + terrain ruggedness) — explicitly deferred: OSM's own Overpass API timed out twice in live testing during this project's own research, and terrain data (Open-Elevation) needs ~4,000 batched calls and caching infrastructure this repo doesn't have yet. - **Duplicate/double-funding detector** (comparing two different nearby sanctioned projects against each other, not the same-project already-funded check that exists today) — explicitly scoped out as a distinct future feature, no code.

Blocked on something outside the code, not a coding task: - eGramSwaraj whole-panchayat GPDP budget plans — CAPTCHA-protected; the human-assisted capture tool exists but was never actually run at scale. - eMARG health facility data — CAPTCHA-blocked, needs a formal NRIDA data request. - NASA GPM IMERG rainfall — needs a free (but real) NASA Earthdata Login nobody has created yet; verified live that the AWS mirror refuses anonymous access. - Satellite disaster confirmation (Planet Labs) — needs a Planet Education & Research application (university email). - Real per-facility health budget data — does not exist publicly beyond NHM's fixed entitlement amounts, which are not actual spending.

Explicitly ruled out, with reasons already investigated: Sentinel satellites (10 m resolution too coarse), Google Earth Engine (approval delay + false-alarm risk), Bhuvan high-resolution imagery (rural areas only get 2.5 m), Mapillary/KartaView/OSM street imagery (no internal village road coverage), MSRTC bus GPS / PWD pothole portal / HMIS (no public data), NITI Aspirational Districts dashboard (doesn't cover either pilot district), AI deepfake detection or AI auto-scoring of complaint severity (a standing decision — photos and text stay evidence for a human officer, never an automatic verdict).

PART 13 — How to run this project today

Reproduced from `README.md` because it is current and correct as of this audit:

```
# 1. Backend API (from backend/) — the frontend expects port 8001
cd backend
python -m uvicorn main:app --host 127.0.0.1 --port 8001

# 2. Frontend (from the repo root), served on localhost so the live camera works
python -m http.server 5500 --bind 127.0.0.1 --directory frontend
```

Page	URL
Officials dashboard	http://127.0.0.1:5500/index.html
Citizen complaint form (live camera)	http://127.0.0.1:5500/report.html
Photo Lab (test any photo against every check)	http://127.0.0.1:5500/photo-lab.html
API docs	http://127.0.0.1:8001/docs

The pothole/crack model (`models/pothole_best.pt`, 150 MB) and the vision-LLM damage grader (needs an OmniRoute-reachable model, `AWAAZIQ_VLM_DAMAGE_MODEL=1` by default) are both optional at the code level — every other check still runs and reports "not available" if either is missing or disabled.

Full data-loading order (22 scripts) and the exact `python -m intelligence.recompute` invocation are in `README.md`'s "Data-loading scripts" section, reproduced in Part 7 of this audit as a source table rather than a run-order list.

PART 14 — Deployment readiness (Vercel + Railway + Supabase)

Added same day as a later pass, after the owner asked to prepare the codebase for a real deployment. None of this existed when Parts 1–14 above were written; this section documents what changed and why, the same way the rest of this audit documents everything else — by reading the actual current source, not by describing an intention.

What would have broken on a naive deploy, found by reading the code before deploying anything: - `backend/database.py` ran a SQLite-only `PRAGMA journal_mode=WAL` / `foreign_keys=ON` event listener and a SQLite-only `check_same_thread` connect arg **unconditionally**, on every connection — both error against a real Postgres `DATABASE_URL`. Now gated on `engine.dialect.name`. - `backend/migrate.py`'s column-migration queries (`PRAGMA table_info`, `sqlite_master`) are SQLite-specific syntax. Now a no-op on any other dialect, since a fresh Postgres database built by `Base.metadata.create_all()` already has every column current `models.py` declares — the migration mechanism exists only for a SQLite file that already has rows and needs a column added after the fact. - Three loaders (`load_gpdp_district_summary.py` / `load_jjm_village_schemes.py` use `bs4`; `load_jjm_water_testing.py` uses `pycryptodome`; several use `python-dateutil`) import real third-party packages that were never declared in `backend/requirements.txt` — a fresh clone would fail on import the first time any of them ran. Added, along with `psycopg2-binary` (the actual Postgres driver) and `pyarrow` (needed to read the flood -inundation loader's parquet source file). - `models/pothole_best.pt` (150 MB, git-ignored, over GitHub's 100 MB limit) has no way to reach a fresh deploy at all. `backend/photo_checks.py :: get_model()` now has an optional one-time download-on-first-use path (`AWAAZIQ_POTHOLE_MODEL_URL`) — point it at a real hosted copy (e.g. a Supabase Storage object URL) and the server fetches it once, automatically, instead of requiring manual file placement. - The frontend's four entry points (`index.html`, `report.html`, `signin.html`, `photo-lab.html`) each independently hardcoded `http://127.0.0.1:8001` as their API fallback across `app.js` / `auth.js` / `report.js` / `photo-lab.js`. Correct for local dev and the single-process ngrok phone demo (same-origin logic already handles that case), wrong the moment frontend (Vercel) and backend (Railway) become two separate hosts. New `frontend/js/config.js` is the one place to set the deployed backend's URL; every page checks it before falling back to its existing local-dev logic, so nothing about local development changed.

The one feature that genuinely cannot run unattended once deployed: C6's vision-LLM call goes through a local OmniRoute gateway on `localhost:20128` — the developer's own machine, not reachable from a Railway container. Rather than rewrite the call path, the real, verified fix is operational, not code: the `omniroute` CLI already supports `tunnel create` (exposes the local gateway publicly) and `tokens create` (a real scoped access token), and reads `OMNIROUTE_BASE_URL` / `OMNIROUTE_API_KEY` from the environment automatically — so setting those two variables on the deployed backend for the duration of a demo makes the existing,

unmodified code work against a tunnelled local gateway, and the existing fallback (already graceful, see Part 8/Part 9) takes over the moment the tunnel or gateway isn't reachable.

New files: `backend/railway.json` (Nixpacks build + start command), `backend/.env.example` (every real environment variable this project reads, documented in place), `frontend/js/config.js`, and `DEPLOYMENT.md` (the actual account-by-account setup steps for all three platforms, including the exact loader-rerun sequence needed to populate a fresh Supabase database — re-running the real loaders rather than migrating the existing SQLite file byte-for-byte, an explicit choice since the loaders are already idempotent and dialect-agnostic via SQLAlchemy).

All four test suites still pass unchanged (291/291 intelligence, 70/70 backend endpoint, 65/65 photo checks, 46/46 pipeline) against the local SQLite setup — every change above is additive and dialect-gated, not a rewrite of the working local path.

PART 15 — Glossary (terms specific to this project's current design)

Term	Meaning here
Asset	A specific, named, geolocated thing (one school, one hospital, one road spot, one water point) with its own full 9-term score — the newer, primary scoring unit, introduced with the village/asset view
Demand cluster	The older scoring unit: a DBSCAN-grouped set of corroborating reports of one category in one area — still live, still what `/map-data` and the priority rail render by default
Village priority	One village's rolled-up view: its score is always its single worst-off asset's score, plus an independent urgent-asset flag
Work group	A sub-cluster grouping (250 m radius) inside a *cluster*, naming the specific spot within it that's broken — the older mechanism, now largely superseded in the dashboard by the asset system but still built and stored every recompute
Data basis	Per-term provenance label (`government_records`, `proxy_...`, or a blended percentage) attached to every score, so a reader can audit which numbers are real and which are estimated
Evidence-only	A real data signal that is shown to a human and never allowed to move a score term directly (flood exposure, hazard corroboration, JJM scheme cost, MGNREGA/PRIASoft finance, GPDP district context)
Name basis / location basis	Provenance flags on every asset: how it got its name (citizen-picked, demo-assigned, register-matched, GeoSadak-matched, unresolved) and where its coordinates come from (register, citizen GPS pin, synthetic seed, village centroid, mixed)
Emergency (urgency)	Since Feature #7, urgency means *only* a graded bridge/building damage signal — never a pothole, staffing gap, or dry tap
Trust-blend	The exponential-decay mechanism that discounts an old government record's weight in favour of a real, corroborated burst of contradicting recent citizen reports
Workstream C	The photo-verification/fake-complaint-defence subsystem (C1–C7)